Group:

Name: Adham amir Shahin **Id: 7968**

Name: Mohamed maher hanafy **Id:8070**

Name: Abdelrahman Shaaban saad **Id:8041**

➢ 8 puzzle game :

■ Node:

. First we did initialization for the node in class node sending parameters of board,parent,move,cost(g), heuristic distance(h) which represent each node of the tree.

■ Puzzle:

Then we initialized our class puzzle which links the nodes to each others

In the get neighbors function, check if the game is solved in the puzzle solved function, get the goal position for each node in get goal position function and calculate the heuristic distance manhatanly and

euclideanly in the heuristic and heuristic_ecl functions.

- **Get_goal_positions** :

Dictionary is returned filled with the value of the goal_state where each value has its position in row and column.

- **Find_neighbors :**

Returns an array called neighbors filled with the valid moves for the zero position in which first of all we try to find the zero position after that we add all the valid moves for this position in the returned array called neighbors.

- **Puzzle_solved :**

Check if the puzzle is solved by comparing the board state by the goal state.

- **Heuristic, Heuristic_ecl :**

It compares the current position of the whole number in the board and return the total distance should be made to reach the goal state using specific equation.

Gamesolving :

We initialized the game solving class which have most of the functions and algorithms.

- **Find path** :

Returns the correct path in an array from where the algorithm started by saving each parent in this array.

- **Solve_bfs** :

Which solves the puzzle using breadth first search:

an algorithm for searching a tree data structure for a node that satisfies a given property. It starts at the tree root and explores all nodes at the present depth prior to moving on to the nodes at the next depth level(explores shallow first).

- **Solve_dfs**:

Which solves the puzzle using depth first search:

an algorithm for traversing or searching tree or graph data structures. The algorithm starts at the root node and explores as far as possible along each branch before backtracking(explores deep first).

- **Solve_idfs**:

Which solves the puzzle using iterative deepening depth first search:

a state space search strategy in which a depth-limited version of depth-first search is run repeatedly with increasing depth limits until the goal is found.

- **Astar, Astarecl**:

Which solve the puzzle starting from the root depending on the cost of each node(g) and the heuristic distance which is the sum of the difference of each node position and its goal position(h) starting from the one having the smallest(g+h) using the priority queue as a data structure.

- **Data structure :**
    1. **Queue**: used in Bfs in which the elements are dequed by the order of adding them to the frontier list so the shallow will be expanded firstly.
    2. **Stack**: used in Dfs in which after the elements is visited its children are pushed in the stack then it is time to pop the last

pushed element in the frontier stack so deepest is expanded firstly.

3. **Priority Queue**: Used in Astar and Astar-Ecl in which the elements with the least F=(g+h) is queued firstly in it.
4. **Set**: used to store the explored nodes in each of the 5 Algorithms.

- **Sample run:**

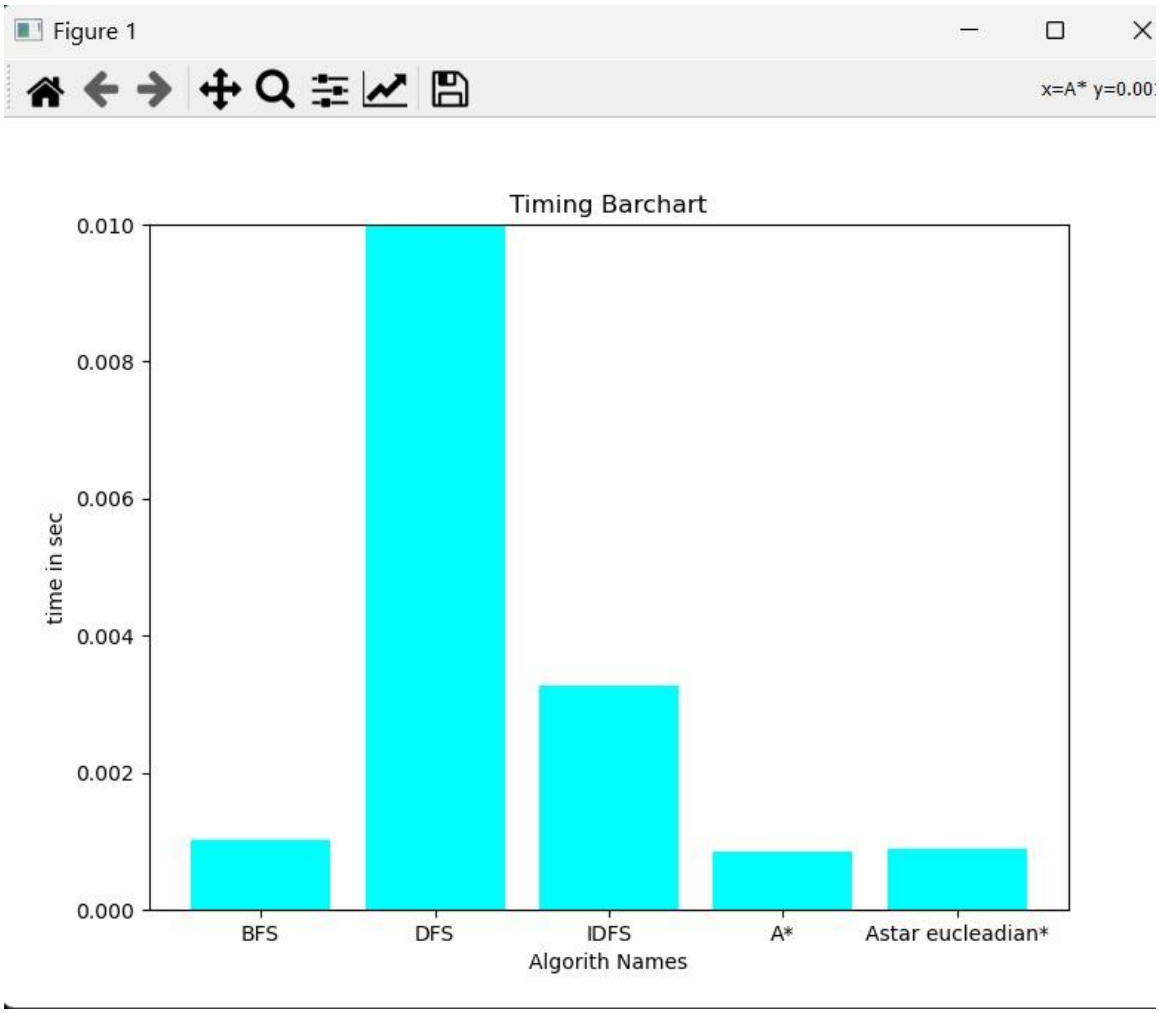
[1,2,5],

[3,4,0],

[6,7,8]

Output :

Timing Barchart:



Total cost:

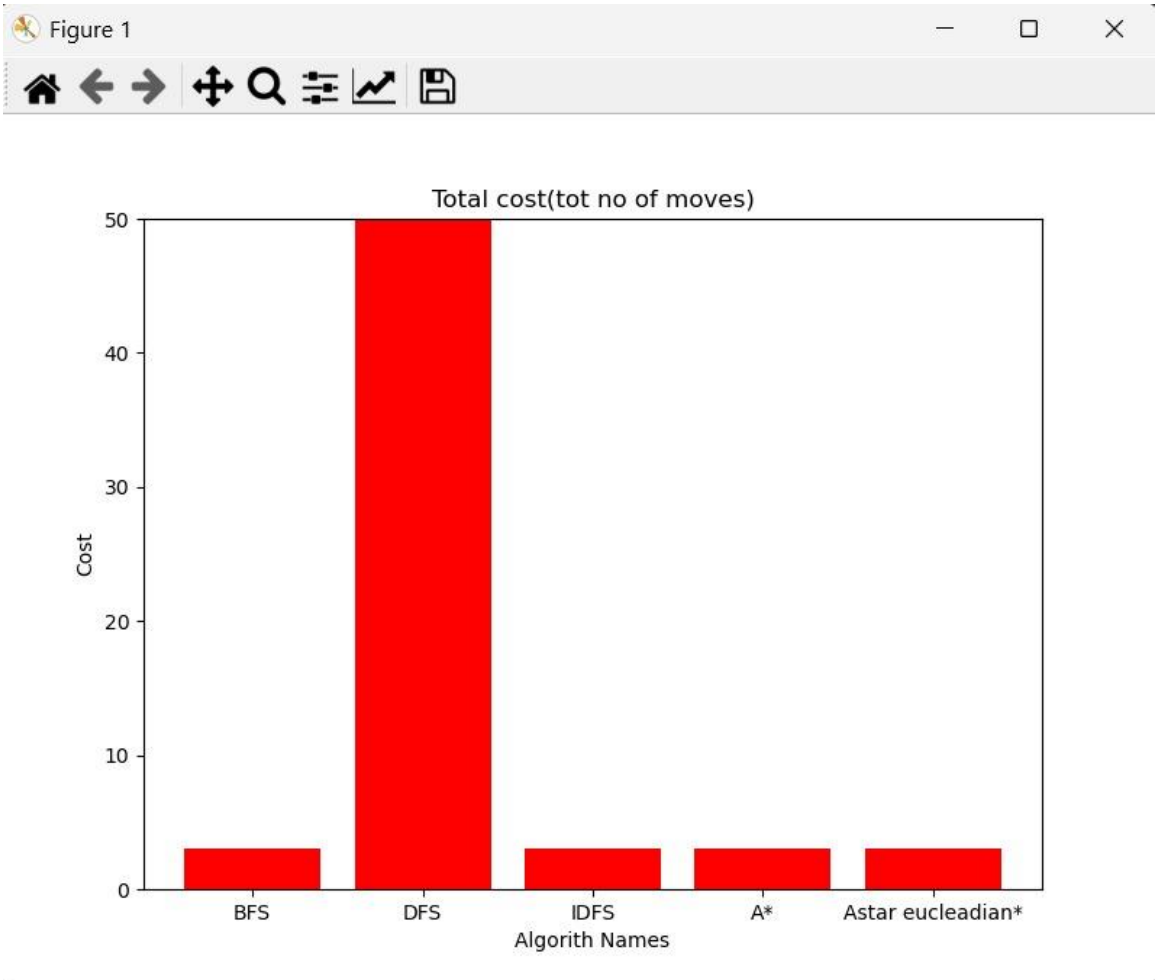
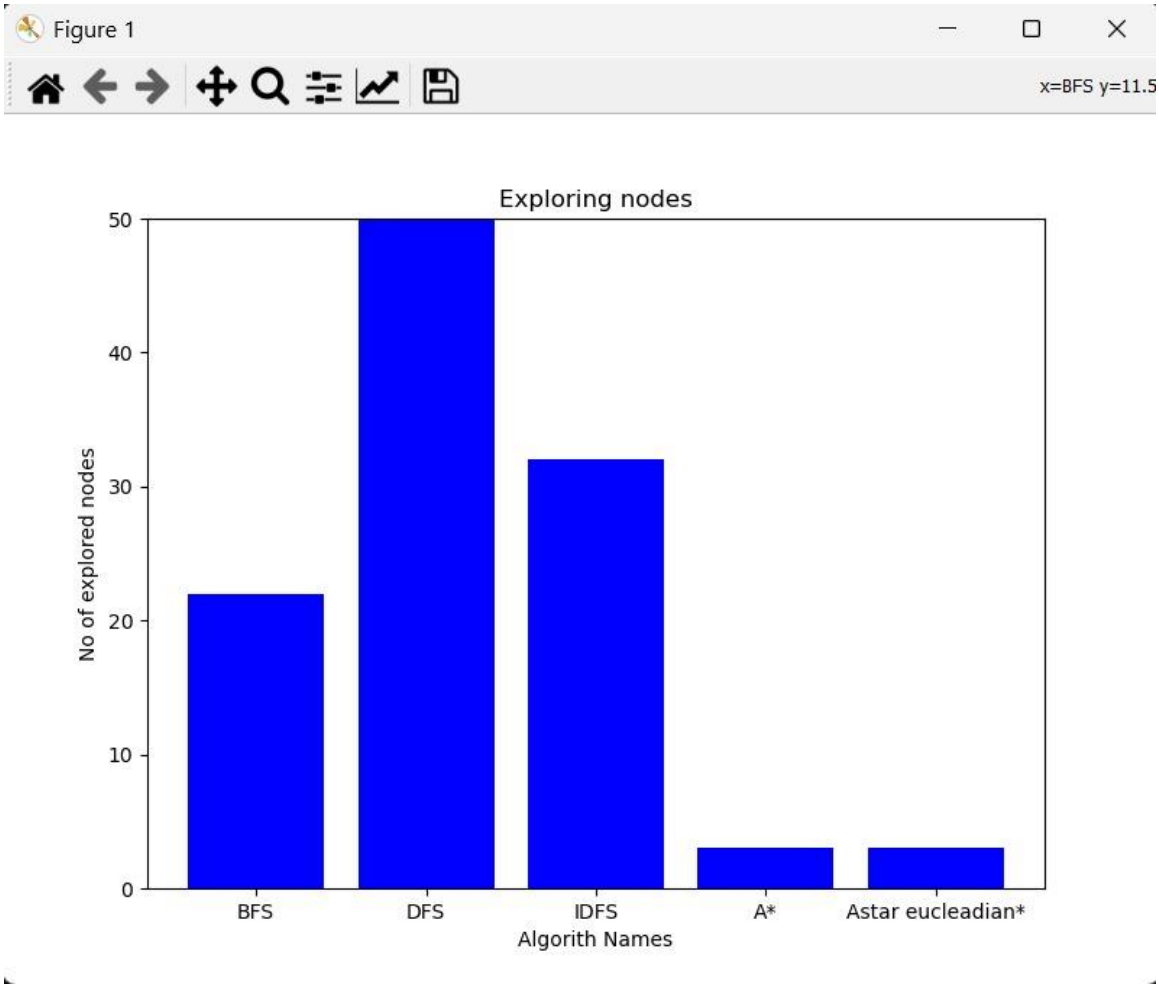


Figure 1
Total cost(tot no of moves)
Cost
50
40
30
20
10
0
BFS
DFS
IDFS
A*
Astar eucleadian*
Algorith Names

Exploring nodes:



Depth of algorithm :